# PROFESSIONAL PENTESTER TOOL CHEAT SHEET

Host discovery • Vulnerability scanners • Credentialed scanning • Web app tools • Throttling & evasion • Automation

**AUTHORIZATION FIRST**

Every tool and flag below assumes a signed scope/rules-of-engagement. Running any of these against systems you are not authorized to test is illegal in most jurisdictions.

This sheet is built directly from — and extends — the vulnerability-scanning lecture material: Nessus scan templates, host discovery, protocol/topology scoping, bandwidth throttling, fragile-system handling, credentialed vs non-credentialed scanning, and automation via CLI/API. Each tool entry has a one-line identity, key flags, a practical command, a "use when" line, and — where relevant — a note tying it back to the lecture.

**Legend:** teal box = tip/best-practice · amber box = scenario/consideration · red box = warning/gotcha.

SECTION 1

# Host Discovery & Network Recon

"Once you do your host discovery... an Nmap scan would be sufficient."

## Nmap

Discovery / Recon

The universal port/service/host mapper

Nmap is the baseline recon tool referenced directly in the lecture — used before a vulnerability scanner even runs, to answer "who's alive, what's open, what's running." It also has limited built-in vulnerability-scanning scripts (NSE).

| Flag / Option | What it does |
|---|---|
| `-sn` | Ping sweep only — host discovery, no port scan |
| `-sV` | Service/version detection on open ports |
| `-sS` | TCP SYN scan — fast, stealthier half-open scan |
| `-p-` | Scan all 65535 ports instead of the top-1000 default |
| `-O` | OS fingerprinting |
| `-T0…T5` | Timing templates — T0/T1 slow & stealthy, T5 fastest/loudest |
| `--script vuln` | Run the NSE 'vuln' script category — Nmap's built-in vuln checks |
| `-oN / -oX / -oA` | Save output as normal / XML / all formats |

**Practical commands**

```
nmap -sn 192.168.56.0/24 # host discovery
nmap -sV -T3 -oN scan.txt 192.168.56.20 # service/version, normal timing
nmap -sS -p- -T2 192.168.56.20 # full port range, throttled
nmap --script vuln 192.168.56.20 # Nmap's own vuln-check scripts
```

**Use when:** First touch on any new scope, and any time you need a fast, no-license second opinion alongside a full scanner.

**Lecture link:** Directly named as the discovery step before vulnerability scanning; also usable as a lightweight vuln scanner via NSE.

## Masscan

Discovery

Internet-scale port scanner

Built for scanning very large ranges extremely fast — trades some accuracy for raw speed compared to Nmap.

**Practical command**

```
masscan -p1-65535 192.168.56.0/24 --rate=1000 -oL masscan_out.txt
# --rate controls packets/sec — this IS your throttle control
```

**Use when:** Very large IP ranges where a full Nmap sweep would take too long.

## Amass / Subfinder

Recon

Attack-surface / subdomain discovery

**Practical commands**

```
subfinder -d client-domain.com -o subs.txt
amass enum -passive -d client-domain.com
```

**Use when:** Before scanning a web-facing scope, to find every in-scope hostname you didn't already know about.

SECTION 2

# Vulnerability Scanners

"A vulnerability scan is going to scan a system and network for known vulnerabilities."

## Nessus (Tenable)

Vuln Scanner

Industry-standard GUI/CLI vulnerability scanner

The scanner shown directly in the lecture. Ships with pre-built templates (Basic Network Scan, Host Discovery, Web App Tests, and current-threat templates like Log4j / ProxyShell / PrintNightmare).

| Setting (from the lecture) | Where it lives / why it matters |
|---|---|
| Scan templates | Host Discovery, Basic Network Scan, dedicated CVE templates (Log4j, ProxyShell, PrintNightmare) |
| Schedule / Repeat / Frequency | Run daily/weekly at a set time outside production hours |
| Targets | CIDR ranges, host lists, or domain names |
| Credentials tab | Add SSH/Windows/DB/API creds to run a credentialed scan |
| Performance options | Throttle back automatically when network congestion is detected |
| Protocol scope | Restrict a scan to just HTTP, just SMTP, etc. |

**Nessus CLI (nessuscli) — local admin tasks**

```
/opt/nessus/sbin/nessuscli update # update plugin feed BEFORE scanning
/opt/nessus/sbin/nessuscli fix --list # list advanced settings
```

**Use when:** Your default full-featured scanner for any engagement — broad and targeted templates, credentialed option, scheduling.

**Rule one from the lecture**

Update the plugin feed immediately before scanning — a scanner "only knows how to scan for things it knows about."

## OpenVAS / Greenbone (GVM)

Vuln Scanner

Free, self-hosted vulnerability scanner

**gvm-cli — scripted scan (GMP protocol)**

```
gvm-cli socket --xml "<get_version/>"
# Typical flow: create_target -> create_task -> start_task -> get_results
# Most teams script this via python-gvm rather than raw XML by hand.
```

**Use when:** Free alternative to Nessus for labs, small shops, or clients who want an open-source stack.

**Lecture link:** Named directly as "try OpenVAS" — same job as Nessus, different vendor.

## Nuclei

Vuln Scanner

Fast, template-driven CVE/misconfig scanner

| Flag / Option | What it does |
|---|---|
| `-u` | Single target URL/host |
| `-l` | File of targets |
| `-t` | Template path/category (e.g. cves/, exposures/) |
| `-severity` | Filter by severity (critical,high,medium,low) |
| `-rate-limit` | Requests per second — your throttle knob |

**Practical command**

```
nuclei -u https://192.168.56.20 -t cves/ -severity critical,high -rate-limit 50
```

**Use when:** Fast, automatable CVE sweeps across many hosts — pairs well with CI/CD or scheduled jobs (see Section 6, Automation).

## Qualys VMDR / Rapid7 InsightVM

Vuln Scanner

Enterprise cloud-managed scanners

Same conceptual job as Nessus/OpenVAS (scan templates, scheduling, credentialed scanning, dashboards) but run at enterprise/cloud scale with asset-risk scoring and long-term trend tracking. You'll meet these on larger client engagements — the workflow you learn on Nessus transfers directly.

SECTION 3

# Web Application Scanners

"Don't forget application protocols like HTTP and SMTP."

## Nikto

Web Scanner

Web-server misconfiguration scanner

Named directly in the lecture as an example of a purpose-built, automatable vulnerability scanner focused on web servers.

| Flag / Option | What it does |
|---|---|
| `-h` | Target host/URL |
| `-p` | Port(s) to test |
| `-ssl` | Force SSL/TLS mode |
| `-Tuning` | Restrict to specific test categories (e.g. 9 = SQLi checks) |

**Practical command**

```
nikto -h https://192.168.56.20 -ssl
```

**Use when:** A fast first pass on any single web server before a heavier full-app assessment.

## OWASP ZAP / Burp Suite

Web Scanner

Full web-app proxy & scanner

**ZAP headless baseline scan**

```
zap-baseline.py -t https://192.168.56.20 -r zap_report.html
```

**Use when:** In-depth manual + automated web-app testing: auth flows, business logic, and intercepting/replaying requests — beyond what a general vuln scanner covers.

## Gobuster / ffuf

Web Recon

Directory & content discovery / fuzzing

**Practical commands**

```
gobuster dir -u https://192.168.56.20 -w /usr/share/wordlists/dirb/common.txt
ffuf -u https://192.168.56.20/FUZZ -w common.txt -mc 200,301,403
```

**Use when:** Finding hidden admin panels, backup files, or unlinked endpoints a scanner's crawler might miss.

## sqlmap

Web Scanner

Automated SQL-injection detection & exploitation

**Practical command**

```
sqlmap -u "https://192.168.56.20/item?id=1" --batch --level=2 --risk=1
```

**Verification, not autopilot**
sqlmap can auto-exploit — per the lecture's core lesson, confirm scope and understand each finding before letting it dump data or escalate.

## WPScan

Web Scanner

WordPress-specific vulnerability scanner

**Practical command**

```
wpscan --url https://192.168.56.20 --enumerate vp,vt,u
```

**Use when:** Any in-scope target running WordPress — plugin/theme/user enum plus known-CVE matching.

SECTION 4

# Credentialed & Internal (AD) Assessment Tools

"If we have credentials, we can act as an internal threat."

## Nessus / OpenVAS credentialed scan

Credentialed

Built-in credentialed mode

Add SSH, Windows (SMB), database, or API credentials directly in the scan's Credentials tab, exactly as shown in the lecture — the scanner then reads patch levels, installed software, and local configuration instead of only external network behaviour.

## CrackMapExec / NetExec

Credentialed / AD

Swiss-army knife for Windows/AD recon

| Flag / Option | What it does |
|---|---|
| `-u / -p` | Username / password (or hash) for authentication |
| `--shares` | Enumerate accessible SMB shares |
| `--sam / --lsa` | Dump local credential material (post-auth) |

**Practical command**

```
netexec smb 192.168.56.0/24 -u jdoe -p 'Password123' --shares
```

**Use when:** Once you have at least one valid credential, to rapidly map where else it works across a Windows/AD environment.

## Enum4linux-ng

Credentialed / AD

SMB/AD enumeration

**Practical command**

```
enum4linux-ng -A 192.168.56.20
```

**Use when:** Pulling users, groups, shares, and password-policy info from a Windows host with little or no credential.

## Impacket suite

Credentialed / AD

Python library + tools for Windows protocols

**Common tools**

```
secretsdump.py DOMAIN/user:pass@192.168.56.20 # dump credential material
psexec.py DOMAIN/user:pass@192.168.56.20 # authenticated remote exec
GetUserSPNs.py DOMAIN/user:pass -dc-ip 10.0.0.1 # Kerberoasting recon
```

**Use when:** Deeper Active Directory verification once you're authorized to confirm impact of a credentialed finding.

## Evil-WinRM

Credentialed

Authenticated Windows remote shell

**Practical command**

```
evil-winrm -i 192.168.56.20 -u jdoe -p 'Password123'
```

**Use when:** Manual verification step after a credentialed finding needs hands-on confirmation, not just a scanner tick-box.

SECTION 5

# Load Balancer, WAF & TLS Tooling

Supporting tools for the topology considerations in the lecture

## wafw00f

Topology

WAF/CDN fingerprinting

**Practical command**

```
wafw00f https://192.168.56.20
```

**Use when:** Before tuning scan speed/aggressiveness — know if a WAF will block or mangle your scan traffic.

## lbd

Topology

Load-balancer detector (DNS + header based)

**Practical command**

```
lbd target.tld
```

**Use when:** Confirming whether "one target" is actually several backend servers before you scope your credentialed scans per-host.

## testssl.sh / sslyze

TLS

TLS/SSL configuration auditing

**Practical commands**

```
testssl.sh https://192.168.56.20
sslyze --regular 192.168.56.20:443
```

**Use when:** Any HTTPS-exposed target — checks weak ciphers, expired/misconfigured certs, protocol downgrade issues that a general vuln scanner may only partially cover.

SECTION 6

# Throttling, Evasion & Fragile-System Handling

"Bandwidth utilization is a huge factor... query throttling helps."

| Tool | Throttle / safety control |
|---|---|
| Nmap | -T0/T1 (slow, stealthy) through -T5 (fast/loud); --scan-delay; --max-rate |
| Masscan | --rate= — directly caps packets per second |
| Nessus | Performance Options → auto-detect network congestion and throttle back |
| OpenVAS | Scan config → max hosts / max checks concurrency settings |
| Nuclei | -rate-limit and -c (concurrency) flags |

### Fragile & non-traditional systems

ICS/SCADA, custom line-of-business apps, IoT, and networked medical devices can crash under normal scan traffic. Per the lecture: identify them up front, exclude them from automated scan ranges, and assess them manually / in coordination with a specialist.

### Detection avoidance vs. politeness

Slowing a scan down serves two purposes at once: it keeps a client's shared bandwidth usable, and — if you're doing true threat emulation — it helps you stay under IDS/IPS flood-detection thresholds.

SECTION 7

# Automation — CLI & API

"Any scanner with command-line options, script it into a workflow."

**Bash — loop a CLI scanner over many targets**

```
while read -r ip; do
nmap -sV -oN "out/${ip}.txt" "$ip"
done < targets.txt
```

**Python — Nessus REST API (skeleton)**

```
import requests
headers = {"X-ApiKeys": "accessKey=...; secretKey=..."}
r = requests.post(f"{URL}/scans", headers=headers, verify=False, json={
"uuid": TEMPLATE_UUID,
"settings": {"name": "Weekly", "text_targets": "192.168.50.20,.30"}})
scan_id = r.json()["scan"]["id"]
requests.post(f"{URL}/scans/{scan_id}/launch", headers=headers, verify=False)
```

**Python — OpenVAS/GVM via python-gvm**

```
from gvm.connections import UnixSocketConnection
from gvm.protocols.gmp import Gmp
# create_target -> create_task -> start_task -> get_results, same pattern as Nessus API
```

| Automation type | Example |
|---|---|
| GUI scheduling | Nessus/OpenVAS scheduled scan — repeat + frequency settings |
| CLI scripting | Bash/PowerShell loop calling nmap/nikto/nuclei per target |
| API integration | Python hitting Nessus/OpenVAS/Qualys REST or GMP APIs |

**Automation ≠ judgement**

Scripts remove repetitive manual steps. They never replace scope decisions, evidence checking, or verification — that stays a human task.

SECTION ★

# Quick-Reference Index

One table, every tool, grouped by job

| Job | Tools |
|---|---|
| Host discovery | Nmap (-sn), Masscan |
| Attack-surface / subdomain recon | Amass, Subfinder |
| General vulnerability scanning | Nessus, OpenVAS/GVM, Qualys VMDR, Rapid7 InsightVM, Nuclei |
| Web application scanning | Nikto, OWASP ZAP, Burp Suite, WPScan, sqlmap, Gobuster/ffuf |
| Credentialed / Active Directory | Nessus/OpenVAS credentialed scan, CrackMapExec/NetExec, Enum4linux-ng, Impacket, Evil-WinRM |
| Load balancer / WAF detection | wafw00f, lbd |
| TLS/SSL auditing | testssl.sh, sslyze |
| Throttling / evasion | Nmap -T/--scan-delay, Masscan --rate, Nessus performance options |
| Automation | Bash/PowerShell CLI loops, Nessus/OpenVAS/Qualys REST or GMP APIs |

**How this maps back to the lecture, end to end**

Scope → Nmap discovery → classify normal/fragile/non-traditional targets → pick protocol/topology-appropriate scan → throttle per bandwidth constraints → run Nessus/OpenVAS (broad, then targeted templates) → layer in web/AD/TLS specialist tools → credentialed re-scan → verify every finding → automate the repeatable parts.

---

*Personal training reference — pair with the companion "Junior Pentester Field Notes" document for the underlying methodology and mindset.*